Routing, and a Typed Agent

Two things were added to the `stalin` game. The sum combinator +, which had been implemented in the library and never used. And a commissariat whose answers are written by a language model.
The interesting result is not either feature. It is that the two could not be joined in the way we intended, and the compiler said why.

1 What the game was missing

The library `lib/algebra.ts` implements all four combinators of Ghani's *Containers for Typed Agentic AI*. The game used three of them. The note that accompanies the game said so twice, and both times called the gap a future project.

The first gap was routing. The tensor ⊠ prompts both sides and requires an answer from each. The product × offers both sides and takes an answer from exactly one. The sum + is different from both. A prompt to a sum already belongs to one side before it is sent, and only that side is ever asked.

The second gap was a typed agent. The claim throughout the note is that a command-line tool is already a container, and that the Claude Code harness is a command-line tool. The game never tested the claim.

2 The sum, and why this decision is one

A player builds tractors from steel, or buys them abroad with gold. These were two independent fields on the autumn order, and both could happen in one year, because they draw on different stocks.

They are not the same road. Steel spent on tractors is steel that armaments also wanted, so the domestic side is a contest and is already a product. Gold spent abroad touches no steel at all, so it contests with nothing. Written as an expression,

```
const build = productC(tractorWorks, armaments, ({ a }) => (a.steel > 0 ? "a" : "b"
    ));
const route = sumC(build, importTractors);
```

The label the trace prints for that composite is its own best description.

```
  ↓ gosplan   ((tractors x armaments) + import)
```

3 The finding

The composite we set out to write was `seqC(commissar, route)`. Prompt the commissar, and let his reply compute which side of the sum to prompt next. It does not typecheck, and it cannot.

The sequential combinator ◁ takes a first prompt together with a function from the first reply to the second prompt. Call that function `next`.

State the constraint precisely, because the loose version of it is false. The right argument of $\triangleleft$ may have as many fibres as it likes. The game already relies on that. Its supply chain is `seqC(transport, trade)`, and the trade container has seven fibres.

```
const supply = seqC(transport, trade);
```

What is constrained is the RETURN TYPE of `next`. The composite's fibres are indexed by pairs, a first shape together with a function landing in one particular second shape, so `next` must have a single fibre's shape as its return type. In the supply chain it does. Whatever the haul reports, `next` returns a `SellGrain` prompt and nothing else, and the seven fibres of trade cost nothing.

A sum is different, and it is different by construction rather than by accident. Its shapes are tagged `left` and `right`, and those tags are what make it a sum. A `next` that chooses between the two therefore returns a union spanning both fibres, and a union of two fibres is not one fibre. Routing is branching, and branching is exactly what `next` may not do.

3.1 What is really going on

The account above is what the compiler checks. Underneath it is a shorter statement, and it is the one worth carrying away.

A container is written here as the union of its fibres, and every combinator is a conditional type, so every combinator distributes over that union. That is the whole trick of the encoding and it is why any of this works at all.

Distributing is only sound when the thing being distributed through preserves the union, and that is where the four combinators part company. Look at what each does with `SB`, the shapes of its second argument.

	shape of the composite	**where `SB` sits**
$\boxtimes$ tensor	`{ left: SA; right: SB }`	a factor of a product
$\times$ product	`{ a: SA; b: SB }`	a factor of a product
$+$ sum	`{ side: "right"; value: SB }`	a factor of a product
$\triangleleft$ seq	`{ first: SA; next: (r: PA) => SB }`	the codomain of a function

For the first three, `SB` is a factor of a product, and Set is a distributive category, so $A \times (X + Y) \cong (A \times X) + (A \times Y)$ and the distribution is an isomorphism. Nothing is lost and the encoding is exact.

For the fourth, `SB` is the codomain of a function type. The functor $(P_A \to -)$ is a right adjoint, so it preserves products and does not preserve coproducts. The distribution is performed anyway, because distributing is the only thing a conditional type knows how to do, and what it produces is not isomorphic to what was written. Applied to a sum, the composite `seqC(a, sumC(b, c))` is computed as

$$(a \triangleleft b) \; + \; (a \triangleleft c).$$

The distribution is performed correctly. The error message shows it happening: the parameter type it reports is a union of two records, one demanding a `next` that returns `left` and the other a `next` that returns `right`.

What routing needs is the other expression.

$$a \triangleleft (b + c).$$

These are not the same. Read them at the level of shapes. The first offers a choice between a function landing in `b` and a function landing in `c`. The second offers a function that lands in one or the other, deciding per reply which. In the notation of ordinary type theory this is the familiar non-isomorphism

$$(P \to B) + (P \to C) \;\ncong\; P \to (B + C),$$

and it fails in the direction that matters. Every function on the left gives a function on the right, and a function on the right that genuinely branches gives neither.

So the sentence to keep is this. Three of the combinators put the second argument's shapes in a product, where distribution is exact. The fourth puts them in a function space, where it is not. What is computed is strictly smaller than what was asked for, and a `next` that routes lives in the difference.

A caveat about how far this claim reaches. The failure in TypeScript is verified, and the compiler output below is the verification. The reading of it as the distributivity failure above is exactly that, a reading. It has not been checked against Appendix B, and it has not been written down in Agda or Lean, which is the obvious next thing to do with it.

The file predicts this in its own comment on `SeqC`.

> *What is lost is a* ***`next`*** *that branches between shapes of* ***`d`****; such a function has a union return type and lies in no single fibre.*

We wrote it down and asked the compiler. Here is the whole of what we asked.

```
const route = sumC(builder, importer);
const advised = seqC(commissar, route);

export const shape = advised.step(
  { tag: "Advise", year: 1931 },
  (r) =>
    r.recommend === "buy"
      ? route.right({ tag: "Buy", gold: 100 })
      : route.left({ tag: "Build", steel: 50 }),
);

export const outcome = await advised.run(shape, 0);
```

And here is what `deno check` answered, trimmed only where it repeats itself.

```
$ deno check spike.ts
TS2345 [ERROR]: Argument of type '{ first: ...; next: (r: ...) =>
  { side: "right"; ... } | { side: "left"; ... }; }' is not assignable to
  parameter of type '{ first: ...; next: (r: ...) => { side: "left"; ... }; }
  | { first: ...; next: (r: ...) => { side: "right"; ... }; }'.
      The types returned by 'next(...)' are incompatible between these types.
        Type '{ side: "right"; ... } | { side: "left"; ... }' is not
          assignable to type '{ side: "right"; ... }'.
          Type '{ side: "left"; ... }' is not assignable to type
            '{ side: "right"; ... }'.
            Types of property 'side' are incompatible.
              Type '"left"' is not assignable to type '"right"'.
export const outcome = await advised.run(shape, 0);
```

```
                                                    ~~~~~
error: Type checking failed.
```

Read where it fails. The call to `step` is accepted. The dependent pair can be built, and it has a perfectly good type. What is rejected is `run`, because that pair is not a shape of the composite container. The construction exists and the composite has no room for it.

This is the limit the note describes in the abstract, met in the concrete. Restricting `next` to a single fibre is exactly what buys the position type back, and the price of that purchase is a `next` that cannot branch.

The consequence is worth stating plainly, because it is a claim about the algebra and not about one call site. Three of the four combinators nest inside each other freely. The fourth carries a side condition, and + is the one operator guaranteed to violate it. So the four are not closed under nesting in this encoding. They are three that compose, and one that composes with everything whose second prompt is already decided.

None of this is a defect in Ghani's construction. In dependent type theory the shapes of a sum are a coproduct, and a function out of a coproduct may branch as much as it likes. The obstruction lives entirely in the encoding, and it was put there deliberately, which Section 6 takes up.

4 What we wrote instead

The agent still chooses the route. The dependency survives as an `await` and a branch, which is what an ordinary program would have written in the first place.

```
const word = await commissar.run({
  tag: "Advise", year, steel: s.steel, gold: s.gold,
  tractors: s.tractors, warReserve: s.warReserve,
}, depth);

const shape = word.recommend === "buy"
  ? route.right({ tag: "BuyTractors", gold: s.gold, year })
  : route.left(build.offer(
    { tag: "BuildTractors", steel: word.recommend === "tractors" ? s.steel : 0,
      plantCapacity: s.plantCapacity, year },
    { tag: "BuildArmaments", steel: word.recommend === "armaments" ? s.steel : 0,
    year },
  ));

const outcome = await route.run(shape, depth);
```

Only the sum is algebraic here. That is worth saying plainly rather than hiding, because the whole argument of the note is that the algebra motivates the architecture and does not verify it. Here is a case where the algebra could not even express what the architecture does.

Name what that costs, because it is not elegance. The composite `seqC(commissar, route)` would have been a *value*. It would have had a label, it could have been passed to another function, and it could have been nested inside a further combinator or announced in the trace as one expression. Four lines of `await` and `if` are none of those things. The composition happens during the run and appears nowhere in the program.

So the property given up is composability, which is the one property a first-class algebra exists to provide. That is too much to give up, and Section 5 gets it back.

5 The combinator that does not distribute

If the trouble is that `seqC` distributes over its right argument's fibres, the repair is a sequential combinator that declines to. It is about forty lines, it lives in `branch.ts`, and it differs from `seqC` in exactly one place.

```
// lib/algebra.ts -- next returns SB, bound fibre by fibre as the
// conditional type distributes over B
export type SeqC<A extends Cont, B extends Cont> =
  A extends Fib<infer SA, infer PA>
    ? B extends Fib<infer SB, infer PB>
      ? Fib<{ first: SA; next: (r: PA) => SB }, { first: PA; second: PB }>
      : never
    : never;

// branch.ts -- next returns Shape<B>, the whole union, and B is
// never distributed over at all
export type SeqBranchC<A extends Cont, B extends Cont> =
  A extends Fib<infer SA, infer PA>
    ? Fib<{ first: SA; next: (r: PA) => Shape<B> },
          { first: PA; second: Positions<B> }>
    : never;
```

The `run` method is identical to `seqC`'s, line for line. Nothing changes at run time. What changes is what may be written down beforehand.

The price is named in the second position. Where `SeqC` has `PB`, the positions of the fibre `next` landed in, `SeqBranchC` has `Positions<B>`, the union of all of them. That is forced rather than chosen. After a `next` that branches there is no single shape left to index by, because which shape it returned is a fact about the run and not about the program. So which side answered becomes a run-time question, which is the same bargain the sum already makes.

What was not paid matters as much. The library is untouched. The five composites that do not route keep their exact indexed positions and pay nothing for this file, and `lib/` stays verbatim from upstream, which is what `lib/README.md` promises. The cost falls only on the composite that needs it.

And the routed decision is a value again.

```
const route   = sumC(build, importTractors);
const advised = seqBranchC(commissar, route);

const { first: word, second: outcome } = await advised.run(
  advised.step(
    { tag: "Advise", year, steel: s.steel, gold: s.gold,
      tractors: s.tractors, warReserve: s.warReserve },
    (r) => r.recommend === "buy"
      ? route.right({ tag: "BuyTractors", gold: s.gold, year })
      : route.left(build.offer(
        { tag: "BuildTractors", steel: r.recommend === "tractors" ? s.steel : 0,
          plantCapacity: s.plantCapacity, year },
        { tag: "BuildArmaments", steel: r.recommend === "armaments" ? s.steel : 0,
    year },
      )),
  ),
  depth,
);
```

It has a label, it can be nested, and the trace prints the whole of it as one expression.

```
↓
 gosplan   (commissar <| ((tractors x armaments) + import))
```

So the developer writes one combinator by hand, once, and everything built with it composes.

6 A sum does not label what comes back

The definition of + in `lib/algebra.ts` carries a comment which matters at the call site. The shapes are tagged `left` and `right`. The positions are not.

```
// The router reads which summand the prompt lies in. Note the position
// is NOT re-tagged: pos (Left p) = c.Pos p, exactly as in Appendix B.
```

So what comes back from `route.run` is the chosen side's own reply with nothing wrapped around it. Which side answered has to be recovered from the reply itself.

```
if ("side" in outcome) {
  if (outcome.side === "a") { ...tractors were built... }
  else { ...steel went to the war reserve... }
} else {
  s.gold = round(s.gold - outcome.goldUsed);
  s.tractors += outcome.tractors;
}
```

The product hands you a tag for free, because a product's position type is a sum and has to say which side it came from. A sum's position type is not, and does not. The caller pays the difference. This is the combinator behaving exactly as defined, and it is still a cost.

6.1 This is the repair, done by hand

Those two paragraphs and Section 3 are the same fact seen twice, which is worth making explicit.

The composite $\triangleleft$ could not build was one whose second position is indexed by a shape chosen at run time. Ask what such a composite would have to look like and the answer falls out. Let `next` return the full union of the right container's shapes, and let the composite's second position be the union of that container's positions. Then the pair typechecks, and the caller is handed a value that could have come from either side.

That is precisely the value the branch above receives, and the test `"side" in outcome` is precisely the discrimination such a caller would have to perform.

Said in the language of Section 3.1, widening the position to a union is how the undistributed expression $a \triangleleft (b + c)$ gets built in a setting that can only compute the distributed one. The runtime test is what stands in for the index the type gave up. It is the same bargain `seqBranchC` makes in Section 5, which is not a coincidence, because they are the same expression and the sum was already paying for it before $\triangleleft$ arrived.

The price of that bargain is the indexed position type, which is the one thing the whole encoding was built to keep. Over a finite index a dependent type is a table, and a table is what TypeScript can write down. Widen the position to a union and the table is gone. What makes this affordable is that the widening is local: it happens inside one combinator, used at one site, and everything else keeps its table.

So the honest summary is not that $\triangleleft$ and + cannot be composed. It is that they can be, at the cost of the property that made either of them worth encoding. We chose to pay it once, at one call site, in four lines of ordinary control flow, rather than in the definition of the combinator where every other use would pay it too.

7 The typed agent

The new server on port 8806 is a commissariat like the other four. It parses a prompt off the wire, dispatches on the tag, and answers. One handler differs.

```
async function askClaude(prompt: string): Promise<string> {
  const cmd = new Deno.Command("claude", {
    args: ["-p", prompt], stdout: "piped", stderr: "piped",
  });
  const { code, stdout } = await cmd.output();
  if (code !== 0) throw new Error(`claude exited ${code}`);
  return new TextDecoder().decode(stdout).trim();
}
```

That is the same `Deno.Command` every other hop in the game is made of. A process, an argument vector, and standard output. From the Plan's side nothing distinguishes this commissariat from the smelter.

The prompt gives him the state and the three words he may answer with. It does not tell him what the game is scored on, and it does not say which answer is wanted.

7.1 Two boundaries, and one of them is real

The reply crosses two boundaries. First from the model's standard output into an `Advice` inside the server. Then from the server's HTTP response into an `Advice` inside the Plan. Both are decoders, and neither is a type check.

The first was tested directly against the kinds of thing a model actually returns.

```
accepted  well-formed      -> {"recommend":"buy","note":"gold is ample"}
accepted  fenced           -> {"recommend":"tractors","note":"the fields need them
    "}
REJECTED  out of union     -> null
REJECTED  note missing     -> null
REJECTED  prose, not JSON  -> null
REJECTED  wrong type       -> null
```

A fenced code block is forgiven, because a model asked for bare JSON sometimes wraps it. Nothing else is. In particular a fourth word fails rather than falling back to a default, because a default would quietly turn an unchecked type into one that looks checked.

The compile-time side of the same rule is asserted in `type-tests.ts`, which is checked and never run.

```
expect<Advice["recommend"]>("tractors");
expect<Advice["recommend"]>("armaments");
expect<Advice["recommend"]>("buy");
// @ts-expect-error  a fourth word is not a thing he can be understood to say
expect<Advice["recommend"]>("potatoes");
```

That file fails if a legal move is rejected, and it fails if an illegal one is accepted, since an unused `@ts-expect-error` is itself an error. We checked that the new assertions have teeth by making the illegal word legal.

```
$ deno check type-tests.ts
TS2578 [ERROR]: Unused '@ts-expect-error' directive.
// @ts-expect-error  a fourth word is not a thing he can be understood to say
error: Type checking failed.
```

8 What it costs

The game was reproducible. A seed played the same game twice, and four separate tools rest on that. The scripted playthrough doubles as a regression test, the strategy search compares outcomes across a grid, and the balance harness does coordinate descent on the constants.

An agent is not reproducible. The same seed was played three times, from a fresh game each time.

```
run 1: imported  |  Foreign tractors bought with our idle gold raise the harvest
    immediate
run 2: imported  |  Gold buys foreign tractors outright, so the harvest rises while
     the 2
run 3: imported  |  Foreign tractors bought with idle gold raise the harvest
    immediately w
```

Three different sentences. The recommendation happened to agree all three times, which is not a guarantee of anything. So the agent is started only by `./up.sh --agent`, and without that flag the Plan does not build the leaf at all. The five-server game is untouched, and we checked that rather than assuming it: the scripted playthrough was captured before the change and after it, and the two are byte-identical.

9 Playing it

```
$ ./up.sh --agent
up: gosplan :8801  agriculture :8802  industry :8803  transport :8804  trade :8805
     commissar :8806
    the commissar is answered by claude; this game is no longer reproducible from
    its seed

$ stalin new --seed 1928
$ stalin sow --labour 90,0,14,10 --procure firm
$ stalin reap --export surplus --buy engineers --build commissar
```

The fourth word on `--build` is the new one. Here is a year decided by the commissar.

```
  1928
  harvest    252.81  (adequate, 2.63/worker)   without tractors: 252.81
  steel           2  (deficit)   tractors +1 (1 imported)   rail +20
  traded         28 gold   grain out 56   steel in 0
  reserve         0  for 1941   engineers 1   plant 20
  dead            1   fields 96 drivers 0 mill 13 rail 10
    - Narkomzem: 86% of quota; wreckers and weather are blamed.
    - Narkomtiazhprom: quota fulfilled.
    - Narkomput: 50% of quota; wreckers and weather are blamed.
    - The Commissar: Gold reserves are ample and foreign tractors raise the harvest
      without consuming the scarce steel, which can then be held in reserve.
```

He took the right branch of the sum. One tractor was imported, and the two units of steel were left alone. The trace shows the composite above the hops it turned into.

```
↓
 gosplan   Reap order={"exportGrain":"surplus","…tradeSt
  ↓ gosplan   ((tractors x armaments) + import)↑
 gosplan   {year, harvest, steel, importedTractors, …}
```

His dispatch sits in the same list as the other three, in the same typeface, worth the same amount. The note already said what that is. Their type guaranteed their shape, and nothing

guaranteed their truth. The difference is that the first three were written by arithmetic a person wrote, and the fourth was not.

10 What was verified

Claim	How
The composite typechecks	`deno check *.ts lib/*.ts`, no errors
The four-word order is validated on the wire	`parseGos` rejects a fifth word
The summands stay apart	`type-tests.ts`, mutation-tested
The decoder refuses bad model output	six cases, four rejected
The five-server game is unchanged	playthrough byte-identical before and after
The sum actually runs	trace shows the label and the branch taken
The routed composite is a value	the trace prints it as one expression
The agent is not deterministic	same seed, three runs, three replies

The feature took an afternoon. The finding is worth more than the feature.

Three combinators nest inside each other freely. The fourth requires that its second prompt be decided by the time the first reply arrives, and routing is the operator that cannot promise that. So the four as given are not closed under nesting in this encoding, and a note that claims the combinators are a language for writing the order down should say so.

What it costs to close them is one combinator written by hand, once, and one position type widened to a union inside it. Composability survives. The routed decision is a value like every other composite in the game.

None of this touches Ghani's construction, where the shapes of a sum are a coproduct and a function out of a coproduct branches freely. It is a property of the encoding, and it was bought on purpose. Over a finite index a dependent type is a table, and a table is what TypeScript can write down. A sum is where the index stops being a single fibre, and $\lhd$ has nothing left to stand on.